Enumerations

A set of constants represented as unique identifiers. Like classes, all **enum** types are reference types. An enum type is declared with an enum declaration. Each enum declaration declares an enum class with the following restrictions:

1. enum constants are implicitly final.
2. enum constants are implicitly static.
3. Any attempt to create an object of an enum type with operator new results in a compilation error.

Example,

```
1  // Fig. 8.10: Book.java
2  // Declaring an enum type with constructor and explicit instance fields
3  // and accessors for these fields
4
5  public enum Book
6  {
7     // declare constants of enum type
8     JHTP( "Java How to Program", "2012" ),
9     CHTP( "C How to Program", "2007" ),
10    IW3HTP( "Internet & World Wide Web How to Program", "2008" ),
11    CPPHTP( "C++ How to Program", "2012" ),
12    VBHTP( "Visual Basic 2010 How to Program", "2011" ),
13    CSHARPHTP( "Visual C# 2010 How to Program", "2011" );
14
15    // instance fields
16    private final String title; // book title
17    private final String copyrightYear; // copyright year
18
19    // enum constructor
20    Book( String bookTitle, String year )
21    {
22       title = bookTitle;
23       copyrightYear = year;
24    } // end enum Book constructor
25
26    // accessor for field title
27    public String getTitle()
28    {
29       return title;
30    } // end method getTitle
31
32    // accessor for field copyrightYear
33    public String getCopyrightYear()
34    {
35       return copyrightYear;
36    } // end method getCopyrightYear
37 } // end enum Book
```

And the main class:

```
1  // Fig. 8.11: EnumTest.java
2  // Testing enum type Book.
3  import java.util.EnumSet;
4
5  public class EnumTest
6  {
7     public static void main( String[] args )
8     {
9        System.out.println( "All books:\n" );
```

```
10
11          // print all books in enum Book
12          for ( Book book : Book.values() )
13             System.out.printf( "%-10s%-45s%s\n", book,
14                 book.getTitle(), book.getCopyrightYear() );
15
16          System.out.println( "\nDisplay a range of enum constants:\n" );
17
18          // print first four books
19          for ( Book book : EnumSet.range( Book.JHTP, Book.CPPHTP ) )
20             System.out.printf( "%-10s%-45s%s\n", book,
21                 book.getTitle(), book.getCopyrightYear() );
22       } // end main
23    } // end class EnumTest
```

```
All books:

JHTP      Java How to Program                           2012
CHTP      C How to Program                              2007
IW3HTP    Internet & World Wide Web How to Program      2008
CPPHTP    C++ How to Program                            2012
VBHTP     Visual Basic 2010 How to Program              2011
CSHARPHTP Visual C# 2010 How to Program                 2011

Display a range of enum constants:

JHTP      Java How to Program                           2012
CHTP      C How to Program                              2007
IW3HTP    Internet & World Wide Web How to Program      2008
CPPHTP    C++ How to Program                            2012
```

Generic Methods

Overloaded methods are often used to perform similar operations on different types of data. **Generic methods** enable you to specify, with a single method declaration, a set of related methods. **We are using arrays of the type-wrapper classes to set up our generic method example, because only reference types can be used with generic methods and classes.**

Example,

```
1  // Fig. 21.1: OverloadedMethods.java
2  // Printing array elements using overloaded methods.
3  public class OverloadedMethods
4  {
5     public static void main( String[] args )
6     {
7        // create arrays of Integer, Double and Character
8        Integer[] integerArray = { 1, 2, 3, 4, 5, 6 };
9        Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7 };
10       Character[] characterArray = { 'H', 'E', 'L', 'L', 'O' };
11
12       System.out.println( "Array integerArray contains:" );
13       printArray( integerArray ); // pass an Integer array
14       System.out.println( "\nArray doubleArray contains:" );
15       printArray( doubleArray ); // pass a Double array
16       System.out.println( "\nArray characterArray contains:" );
17       printArray( characterArray ); // pass a Character array
18    } // end main
19
20    // method printArray to print Integer array
21    public static void printArray( Integer[] inputArray )
22    {
```

```
23         // display array elements
24         for ( Integer element : inputArray )
25            System.out.printf( "%s ", element );
26
27         System.out.println();
28      } // end method printArray
29
30      // method printArray to print Double array
31      public static void printArray( Double[] inputArray )
32      {
33         // display array elements
34         for ( Double element : inputArray )
35            System.out.printf( "%s ", element );
36
37         System.out.println();
38      } // end method printArray
39
40      // method printArray to print Character array
41      public static void printArray( Character[] inputArray )
42      {
43         // display array elements
44         for ( Character element : inputArray )
45            System.out.printf( "%s ", element );
46
47         System.out.println();
48      } // end method printArray
49   } // end class OverloadedMethods
```

Here is the example using generic Method

```
1  // Fig. 21.3: GenericMethodTest.java
2  // Printing array elements using generic method printArray.
3
4  public class GenericMethodTest
5  {
6     public static void main( String[] args )
7     {
8        // create arrays of Integer, Double and Character
9        Integer[] intArray = { 1, 2, 3, 4, 5 };
10       Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7 };
11       Character[] charArray = { 'H', 'E', 'L', 'L', 'O' };
12
13       System.out.println( "Array integerArray contains:" );
14       printArray( integerArray ); // pass an Integer array
15       System.out.println( "\nArray doubleArray contains:" );
16       printArray( doubleArray ); // pass a Double array
17       System.out.println( "\nArray characterArray contains:" );
18       printArray( characterArray ); // pass a Character array
19    } // end main
20
21    // generic method printArray
22    public static < T > void printArray( T[] inputArray )
23    {
24       // display array elements
25       for ( T element : inputArray )
26          System.out.printf( "%s ", element );
27
28       System.out.println();
29    } // end method printArray
30 } // end class GenericMethodTest
```

```
Array integerArray contains:
1 2 3 4 5 6

Array doubleArray contains:
1.1 2.2 3.3 4.4 5.5 6.6 7.7

Array characterArray contains:
H E L L O
```